AutoNex Engineering Logbook

WRO 2026 — Future Engineers — Self-Driving Cars

Team	AutoNex
Institution	Instituto Tecnológico de Las Américas (ITLA)
Repository	https://github.com/Eddym06/AutoNex
Document	Engineering logbook — summary version
Date	November 13, 2026 (data closed on the 12th; CAD uploaded on the 13th)



> This document summarizes the design and development of AutoNex between November 6 and 12, 2026. Software was developed in four stages: structure and Ackermann chassis (6–7), parking and localization (8–9), motor control, persistent map and trajectory planner (10), and fault closure (11). Electronics was documented in parallel, and mechanics arrived in two deliveries: v1 on October 16 and v2 on November 12. Every figure carries its source: software figures come from simulator tests and commits, electronics figures from datasheets and bench work, and mechanics figures from the CAD and from measurements by Sebastian Josue Diaz Encarnacion. Anything not measured on the car is marked as an **assumption** or **pending**.

1. Executive summary

AutoNex is an autonomous Ackermann-steering, rear-wheel-drive car built around two ESP32 boards. The track is drawn anew every round, so a recorded trajectory is useless: the car has to read the track, store it and repeat it quickly.

The first lap is driven slowly, with LiDAR and camera, to build the map. On the second and third laps the Intelligent Trajectory (IT) planner takes over and finds the fastest line over that map. At the end, the car parks in the bay with an Ackermann maneuver using two direction changes.

By November 12 the software was validated in simulation: **24 out of 24 rounds, 62 points and no contacts** on feasible tracks, with a 118.9 s average. The simulator is not a separate model: it compiles the same firmware C code to WebAssembly.

Electronics is documented and assembly is progressing block by block. Mechanics follows v2, with a differential and rear bearings; v1 was tested and slipped at the rear. v2 has not been tested yet. Put plainly: **the full mission has not yet run on the real car**.

2. Team and responsibilities

Role	Name	Responsibility
Software	Eddy Manuel Piantini Martínez	Perception, localization, planning, control, simulator and testing
Mechanics and design	Sebastian Josue Diaz Encarnacion	Chassis, Ackermann steering, mounts, CAD and weights
Electronics	Yeremy Manuel Bautista Portes	Power, wiring, drivers, sensors and commissioning

Each area has an owner, but some decisions do not fit in one area alone. The maximum steering angle is set by mechanics and limits the line the planner may request. Servo current is measured by electronics, which decides whether the 5.1 V bus holds or a separate branch is needed. When one of those discussions changed something, it is recorded here.

3. The problem and the strategy

3.1 What the category requires

- Ackermann steering, with no turning by speed difference between wheels.
- Maximum dimensions 300 × 200 × 300 mm and maximum mass 1.5 kg.
- Three laps, passing red signs on the right and green signs on the left.
- Final parking inside the bay, with the car inside and aligned.
- No wireless communication during the round, and an original repository.

The v2 differential is mechanical, not a two-motor differential drive, but the word appears in the rules and it is worth confirming with the official text that it is allowed.

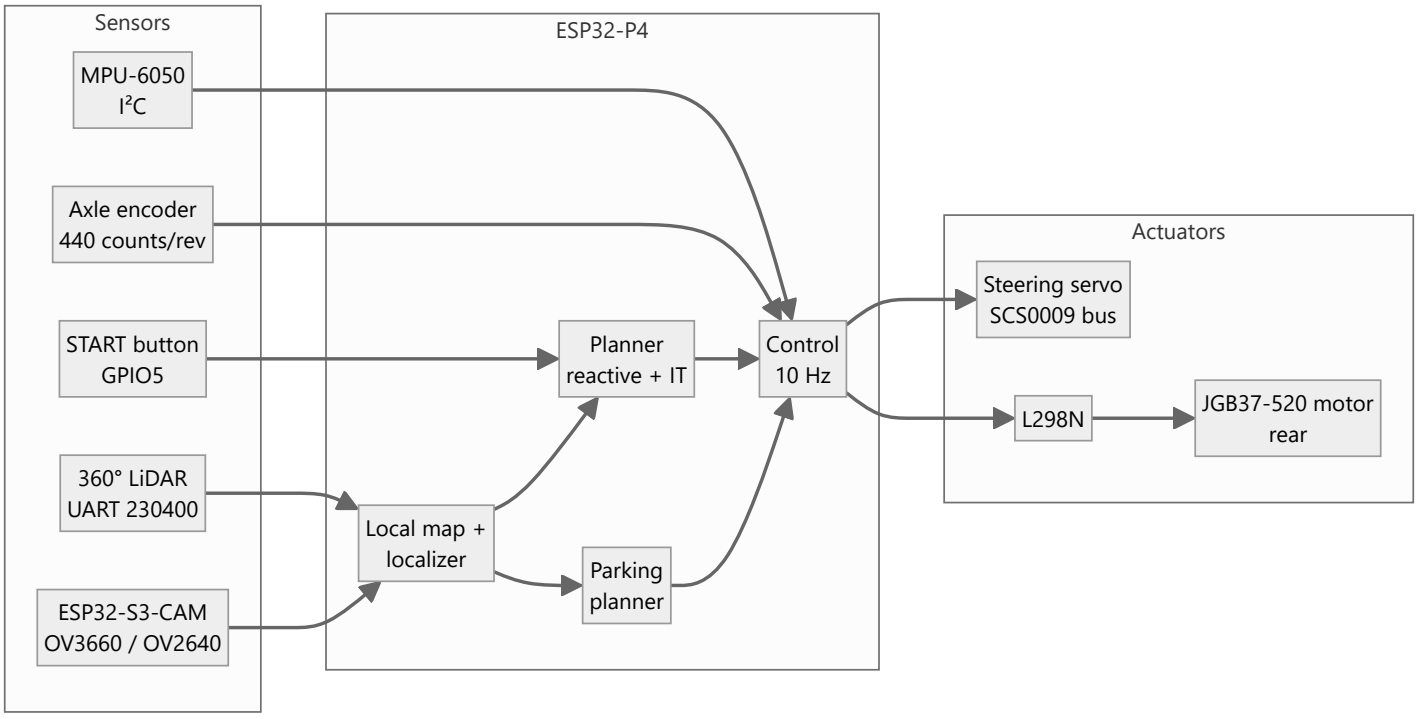
3.2 Simulate before soldering

The track changes every round and the chassis was not finished yet. Instead of waiting, we built a simulator that runs the real firmware: the same C code compiled to WebAssembly, with LiDAR scans, synthetic vision tables and IMU, noise and latency included. That let us close the mission logic, compare configurations and find errors that would have cost an afternoon in the workshop.

It also has a limit worth stating out loud: until the car drives, all of this is simulation. Grip, weight distribution, servo delay and motor speed are editable values, not measurements.

4. Architecture

4.1 Hardware



4.2 Software

The P4 loop is cooperative and takes about 1 ms. Every decision happens **once per LiDAR scan** (~10 Hz): the local map is rebuilt, the pose is updated, the command is chosen and the actuators are written. Between scans, the IT fast loop propagates the pose at 50 Hz using encoders and gyroscope. The IMU is sampled at 400 Hz and integrated at 100 Hz. A 300 ms watchdog sets the command to zero if scans or gyroscope data stop arriving.

The flow is: LiDAR and camera feed the local map; the localizer fits the pose against the walls; `trace_map` stores the field; IT or the reactive planner chooses the line; parking takes over at the end; and MIC or PWM translates the command into motor and servo action.

4.3 Phase machine

The firmware goes through six phases: `IDLE` waits for the button; `RUN` drives the three laps, with the reactive planner on the first and IT on the following ones; `PARK_SEARCH` looks for the bay against the outer wall; `PARKING` performs the two-direction-change entry; `PARK_EXIT` leaves the bay if the car starts inside; and `DONE` stops the car once the goal is complete. Safety is the same in every phase: if the LiDAR or gyroscope fails, the speed command goes to zero. Starting is physical (`GPIO5`); the web page can stop the car but cannot start it.

5. Work log

Software and electronics dates come from commits. Software is signed by Eddy Manuel Piantini Martínez and electronics by Yeremy Manuel Bautista Portes. Sebastian Josue Diaz Encarnacion has no commits: his dates come from the parts manuals and his report.

5.1 Before the repository — v1 mechanics (October 16)

- First chassis, v1: yellow dual-shaft DC motor, 300 × 38 × 150 mm main part and eight documented parts.
- v1 test: it slipped and skidded at the rear. The axle had no bearings and steering travel was short (70° programmed in the servo).
- Intermediate stage: JGB37-520 with a 1:1 two-gear transmission. The axle received the same force and slip remained unchanged.

- Next attempt: differential, rear-drive bearings and a motor with an encoder.

Evidence: `AutoNEx_V1.pdf` and `Informe_Mecanica_AutoNex_Sebastian.md`. There is no date or test measurement for the intermediate stage.

5.2 November 6 and 7 — start, structure and Ackermann chassis

- The repository and the team and vehicle README are created. The category, goal and three-stage strategy are defined. The repository is written in English and internal documentation in Spanish.
- The structure is organized: P4 firmware, S3 vision, motor control, shared protocols and simulator.
- The vehicle software is added and the simulator and control are adapted to the **Ackermann chassis**: a single rear motor, steering servo and bicycle geometry.
- It is documented that the adaptation is incomplete and that earlier results do not validate the new configuration.

Evidence: four commits on November 6 and 7. The old code mixed both wheels as if there were a differential; under Ackermann, `hw_motors_drive` drives only the left L298N channel.

5.3 November 8 and 9 — parking and localization

- The bay is dimensioned to the chassis and the 40° Ackermann maneuver is implemented. An S-shaped entry with a direction-change penalty is tested, then replaced by a **three-segment maneuver with two direction changes**.
- The entry is measured with an in-house tool and the diagnosis is documented.
- The servo is set **before** moving the car: it takes about 0.2 s to travel from one stop to the other.
- The entry starts where the exit ends, and arcs are cut by **gyroscope heading**. The measurement is filtered by direction and seed.

Evidence: seven commits on November 8 and 9. Result: the exit takes 1 pass and 8.1–8.6 s; the entry drops to 3–5 passes, still short of the target of 3.

5.4 November 10 — MIC, persistent map and intelligent trajectory

- **MIC (Motor Intelligent Control).** Central traction control with a physical JGB37-520 model, a third-order observer and a look-ahead planner. Two braking modes: progressive and aggressive, the latter limited by the 2 A of the L298N.
- `trace_map`**.** Persistent 4 × 4 m map in 2.5 cm cells. It stores free space, colored objects with position, and the parking bay. It is saved to flash at the end of the round.
- `arena_localizer`**.** Pose fitting against 8 wall segments with Gauss-Newton and Huber weights. Mean error 2.3 cm and maximum 3.5 cm in simulation.
- **IT.** Optimization planner using the chassis footprint (26 × 13 cm), friction circle, rear weight distribution, 40° steering limit and servo speed. Margins grow with speed, and grip is learned lap after lap.
- IT is integrated into the mission and the professional README is published.

Evidence: eleven commits on November 10. Problems found that day: `odometry.c` discarded samples above 150 cm/s and the limit was raised to 300; the follower went from 10 to 50 Hz; and `race_line` was left implemented but disabled. IT result: 12 of 12 rounds with 62 points; with MIC, 7 rounds, 62 points, 0 contacts and a 151 cm/s top speed.

5.5 November 11 — 600 rpm, fault closure and 24 out of 24

- The whole repository moves to the **600 rpm** motor currently mounted. The final motor will be 1000 rpm and is selected with a profile. Mapping speed is set to **34 cm/s**.
- A parking approach speed limit (23 cm/s) is added, and the entry chooses among **5 candidate plans**.
- Two faults are closed: the 120 s stall at the lap-3 corner (93° heading error treated as "behind") and the contact with the parking stop (margin from 0.5 to 0.75 cm).
- The measurement tool now uses **feasible** tracks by default; the historical "1 in 3 ends in `wrong-side`" was a draw error. Parking no longer stays `stalled`, using the nearest free pose.

Evidence: six commits on November 11. Result: **24 rounds, 62 points, 0 contacts and a 118.9 s average**; 105 parking passes over 24 rounds (4.4 average) and a 16–17 s entry.

5.6 November 12 — electronics, CAD and this logbook

- Electronics is updated with the servo communication board and fully translated into English in `docs/ELECTRONICS.md`.
- The eight images are renamed and uploaded; the folder ends up at `photos/`, in the repository root.
- Sebastian Josue Diaz Encarnacion delivers the **v2** parts manual (26 SolidWorks files: 24 parts and 2 assemblies), with a 260 × 120 × 45.65 mm chassis, differential and rear bearings, plus his correction report.
- This logbook is started. On the 13th, `CAD_File/` is uploaded with 30 parts and assemblies (47.7 MB).

Evidence: the servo adapter, the electronics translation, the four photo commits and the CAD. v2 **has not been tested** yet.

6. Software in detail

6.1 Perception, local map and localization

- The local map is a **160 × 160 grid of 2.5 cm cells** (4 × 4 m) rebuilt on every scan with Bresenham raycasting. The camera contributes up to **12 objects** with color, center and depth; if LiDAR does not confirm the return, they expire.
- `arena_localizer` fits the pose against 8 wall segments with Gauss-Newton and Huber weights, rejecting jumps larger than 30 cm or 8°. `park_localize` uses the bay wall, its opening and the stops.
- Odometry combines wheel angle and gyroscope heading, detects slip and resets after a sample gap longer than 0.5 s. The controller advances the pose by the LiDAR latency (`0.10 s`).
- Known limitation: on hardware, returns at 0 mm or > 12 m are discarded and `blind` never activates. An obstacle against the nose simply does not appear; that branch exists only in the simulator.

6.2 `trace_map`

- 4 × 4 m grid with 2.5 cm cells in field coordinates. Each lap adds weighted returns, so repeated passes **average** instead of thickening the map.
- It stores objects with stable id, position and color, plus the parking bay detected by geometry. It is written to a flash partition at the end of the round, not during it.
- Status: storage is written but **not tested on the P4**. Map accuracy at 34 cm/s is not measured either.

6.3 Intelligent Trajectory (IT)

- Minimizes lap time over the free space left by the pillars, respecting red → right and green → left.
- It knows the car: rear weight (62 % assumed), friction circle, 40° steering limit, servo speed and a 26 × 13 cm footprint.
- Margins grow with speed, and assumed grip increases only while the car runs clean. The 50 Hz fast loop propagates the pose between scans.
- Cost: ~0.4 s per plan on a PC. On the P4 the time is not measured, so the plan is sliced into ~10 ms chunks.

6.4 MIC (Motor Intelligent Control)

- Physical JGB37-520 model: back-EMF, friction, battery, L298N drop and current limit. A third-order observer predicts between encoder counts (0.047 cm per count) and corrects with the model.
- Look-ahead planner with two braking modes: progressive without reverse voltage, and aggressive with reverse voltage, limited by the 2 A of the bridge.
- Status: tested only against a simulated motor, integrated in `hw_motors.c` behind `CONFIG_WRO_USE_MIC` and **disabled**. Assumed parameters: resistance 2.6 Ω, mass 1.2 kg, dry friction 0.6 N and viscous friction 0.5 N·s/m.

Test with simulated motor	Result
300 cm route with a curve and parking	300.8 cm in 8.8 s (vs. ~11.4 s with conservative ramps); 0 s of reverse voltage; 2.0 A peak
Same route with R +15 %, mass +30 %, friction +40 % and low battery	Equally accurate, 0 s of active braking
Stop from 100 cm/s	Progressive 38.6 cm / 1.12 s; aggressive 25.4 cm / 0.52 s

6.5 Parking

- Pose A* on a 121 × 71 × 36 grid (1 cm, 10°), 3.5 cm primitives and `steer ∈ {-1, 0, +1}`. Cost: 3.5 cm per step (1.1× in reverse) + 12 cm per direction change.
- Entry goal: four corners inside the bay with 0.5 cm clearance and $|\theta| \le 4°$; exit with $|\theta| \le 10°$.
- The normal entry is **symmetric** to the exit: cross the lane up to 2 cm ahead and 1.5 cm farther from the wall than the pose where the exit ended.
- Five candidate plans are used if the arrival pose has no plan; if the measurement overlaps the bay, the nearest free pose is used (up to 2 cm and 6°). Arcs are cut by gyroscope heading, and steering is applied with the car stopped.
- Result: 2–5 passes (~4 average) and 16–17 s; 0 failures on seeds 1–8. On seeds 9–32, 5 failures out of 56 (9 %, previously 17 %).

6.6 Reactive control and safety

- The reactive planner evaluates **343 paths** of three 20 cm arcs and scores them by clearance, lateral deviation, heading, curvature, advance and "step". Three-layer hysteresis: 2-point tie band, preference for the previous action and, on a tie, the smoothest turn.
- If there is no path, it performs an escape maneuver and, if that is exhausted, stops. Pivot in place is used after 2 s without a path and a heading deviation ≥ 15°, if there is room.
- The 300 ms watchdog sets the command to zero if LiDAR or gyroscope data is missing. It does not detect a disconnected encoder by itself: that is still pending. Starting is physical (`GPIO5`); the web page can only stop the car.

6.7 Simulator, video and limits

- The simulator runs the same `wro_core.wasm` as the firmware. The world advances with `dt = 0.01 s`, 8 ms process delay and 12 ms command delay.
- Bicycle model with a first-order servo (420°/s, $\tau = 0.045$ s), motor with $\tau = 0.16$ s, maximum acceleration 180 cm/s², gain 0.94, burst slip and sliding collision.
- Measured cost: 1.5 ms per step in the browser; 4× is sustainable, 8× is not. Automated tests cover missions, IT, parking entry, odometry, localizer, `trace_map`, MIC and a 100-track bench.

A full round was recorded on November 10: youtu.be/XHigaxjfDUE. The local copy is at `videos/simulator-video-nov-10.mp4` (2.7 MB). It is simulation, not a track test; the v2 chassis has not driven yet.

Things that can work in the simulator and fail in the car: real actuators receive commands in 100 ms steps; the servo has a configurable stop rather than a ramp; the real car has a single encoder while the simulator averages both wheels; the real LiDAR does not emit `blind` or points closer than 5 cm; the hardware limits speed to 40 cm/s; and it waits for 50 quiet gyroscope samples before starting.

6.8 Discarded decisions

Idea	Why it was discarded
Historical racing line (`race_line`)	With 7 signs it did not save time and reduced reliability
Peeking at uncolored pillars (`peek`)	Reduced perfect tracks from 95 to 89 on the bench
Clearance braking (`tight_clear`)	Cost ~18 s and broke parking
Three-segment maneuver	The steering pattern was inverted and, once corrected, found no solution
Two-arc S from the lane	The rear corner catches the stop at 6.5 cm
1000 rpm motor for today	The mounted motor is 600 rpm; the 1000 rpm profile is ready

7. Electronics in detail

7.1 Components and rationale

Component	Choice	Reason
Battery	3S LiPo, 11.1 V, 2200 mAh, 50C (HOOVO, 174 g)	Single source; EC3 on the battery and XT60H on the car, with an adapter lead
Converter	XL4015, 5 A, non-isolated	Steps down to 5.1 V; input and output share GND
H-bridge	L298N	Forward and reverse; the IRF520 is unidirectional
Motor	JGB37-520, 12 V, 600 rpm today / 1000 rpm final	Built-in quadrature Hall encoder, 440 counts/rev
Servo	SCS0009 bus servo	Metal gears and real position feedback
Servo adapter	Waveshare Bus Servo Adapter (A), SKU 25514	Half-duplex, 3.3 V levels and protected terminal block
IMU	MPU-6050 on GY-521	Z-axis gyro; onboard regulator and pull-ups
LiDAR	D500/D6, 360° time-of-flight	12 m, 10 Hz, 230400 baud; works under strong ambient light
Main board	ESP32-P4	32 MB PSRAM, two cores, console USB
Vision	ESP32-S3-CAM with OV3660/OV2640	Soldered camera, PSRAM and FPC extension to move the module to the nose

7.2 Power

- The battery enters through the XT60H (EC3 with adapter lead), passes through a **5 A** fuse and the main switch. From there two branches leave: one direct to the L298N (12.6 V) and one to the XL4015 (5.1 V).
- The 5.1 V bus feeds both boards, the LiDAR, the IMU, the L298N logic and the servo adapter. The adapter does not regulate: it delivers whatever enters `DC+`, and 5.1 V is within the 4 to 7.4 V range of the SCS0009.
- The adapter carries a **470 µF** capacitor between `DC+` and `DC−` to absorb the servo peak. All grounds go to a single point: a floating GND between motor and signal turns into garbage data.

5.1 V bus load	Typical	Peak
ESP32-P4	350 mA	550 mA
ESP32-S3-CAM	180 mA	300 mA
LiDAR	290 mA	320 mA
Servo	150 mA	1000 mA
Adapter, L298N logic and IMU	49 mA	64 mA
Total	**≈ 1.0 A**	**≈ 2.2 A**

The XL4015 is rated at 5 A, but without a heatsink it should not exceed 2.5 A. Including the motor, the battery delivers at most ~3.5 A peak, well below what a 2200 mAh 50C pack can supply.

7.3 Motor and steering

Motor. It uses a single L298N channel. Commands go through `IN1` and `IN2`; the `ENA` jumper stays in place and PWM is applied to the inputs. The module's 5 V jumper is removed and the logic is powered from the XL4015.

IN1	IN2	Action
PWM	0	Forward
0	PWM	Reverse
0	0	Free
1	1	Hard brake; not used to stop

Encoder. The six-pin connector is silkscreened `M1`, `GND`, `C1`, `C2`, `VCC`, `M2`. The vendor schematic does not match, so it is checked with a multimeter: `M1` and `M2` are the only pair with low resistance. The channels use 4.7 kΩ pull-ups to 3.3 V.

Servo. The SCS0009 speaks over a single half-duplex wire. The Waveshare adapter separates TX and RX, keeps the response at 3.3 V and distributes power. Both jumpers go in row `A` (UART). `TX` goes to `GPIO26` and `RX` to `GPIO21`, without crossing them: RX to RX and TX to TX. The adapter does not regulate servo voltage; if the `PWR` LED does not light, the solution is to power only the adapter from a 6 V branch, not to raise the whole bus.

7.4 P4 pin map

Function	GPIO	Note
Motor `IN1` / `IN2`	32 / 33	1 kHz PWM
Servo TX / RX	26 / 21	To adapter, 3.3 V
Encoder A / B	27 / 23	4.7 kΩ pull-up
LiDAR RX	22	230400 baud
S3 link RX / TX	38 / 37	From `GPIO1` and to S3 `GPIO14`
IMU SDA / SCL	7 / 8	I^2C, 0x68
START button	5	To GND, internal pull-up
Battery (future)	20	ADC, divider pending

Free pins: `2`, `3`, `4`, `6`, `46`, `47`, `48`. Do not use `24` / `25` (USB OTG/JTAG), `0`, `34`, `35`, `36` (boot) or the audio, microSD, Ethernet and LED pins of the board.

7.5 Camera: mounting and flex extension

The S3-CAM does not sit where the camera sits. The v2 CAD reserves the nose for the camera module and leaves the board at the rear, with the rest of the electronics. To separate them we use a **24-pin, 0.5 mm FPC extension**, male-to-female, about 20 cm long: the module stays alone at the front and the S3-CAM at the rear, without moving the board connector.

The extension is the same type as the original flex: 24 contacts, 0.5 mm pitch and contacts on the same side. A Raspberry Pi Zero Camera flex does not work (22 pins, MIPI CSI-2): that module uses a different signal and connector.

The length is short on purpose. The camera runs over DVP, eight parallel bits, and at 20 MHz a long flex behaves like an antenna: shifted pixels and broken frames appear. 20 cm should be enough; if errors appear, the first adjustment is to lower the camera clock, not to lengthen the cable. Before mounting it, check that the connector seats without force, that the image arrives complete and that the flex is not taut when the chassis is closed. In the CAD the camera is at the front with no defined height or tilt; those two dimensions are measured with the car assembled.

7.6 Commissioning

1. Without the battery, check that 12 V and 5 V have no continuity to GND. Power the XL4015 from a 12 V lab supply and set it to 5.1 V with no load; repeat with a 10 Ω resistor and measure again at the board pin.
2. Power the board alone and confirm that it boots; connect the LiDAR and verify that it spins and delivers data; connect the IMU and check 0x68.
3. Power the S3-CAM and check the 921600 link without broken frames; turn the motor shaft by hand and confirm that the encoder counts in both directions.
4. Before the servo: jumpers in `A`, 5.1 V between `DC+` and `DC−`, 5.1 V between `V` and `G` and the `PWR` LED on. Only then connect the servo and move it with the wheels off the ground.
5. Finally the motor, off the ground and at 10 % PWM.

If something fails, disconnect the last thing added and step back. That way the error shows up where it is, not somewhere among the twenty cables of the car.

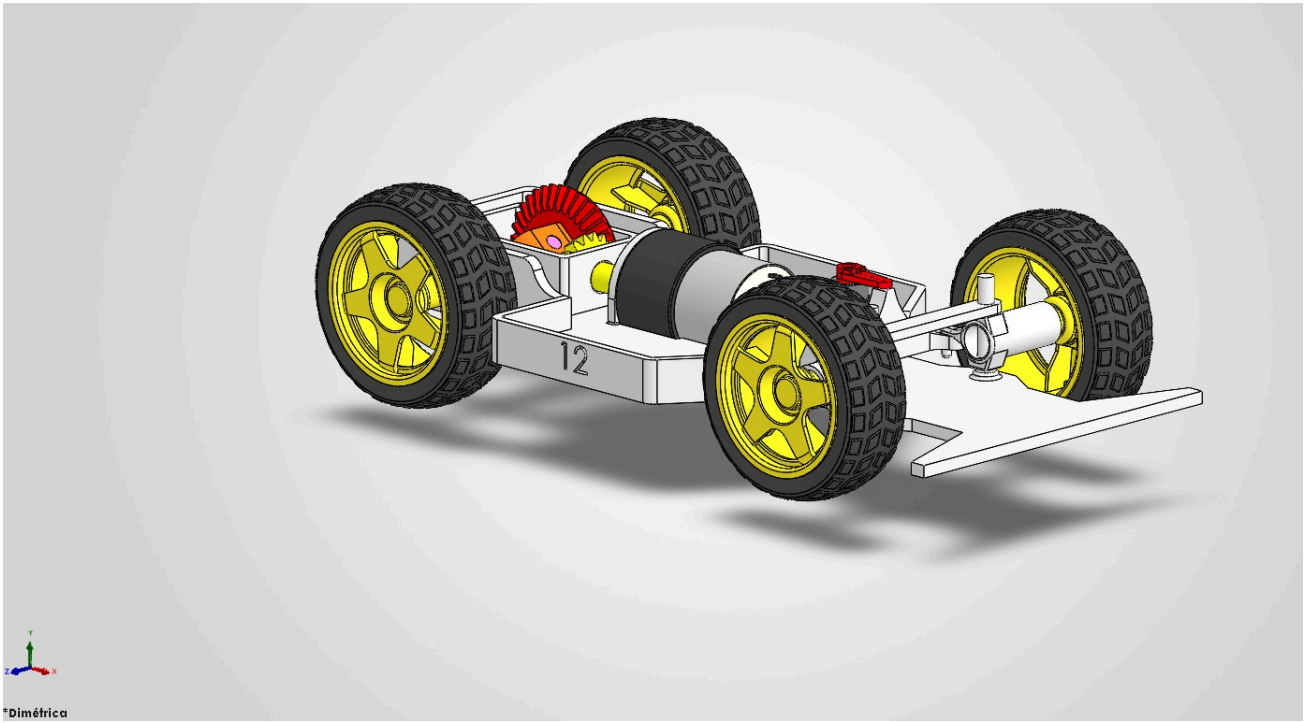
7.7 Electronics pending items

- Measure real motor current with the complete car; the 2.5 A stall figure is from the datasheet.

- Test the servo at 1 Mbit/s with the adapter and the real cable, with the servo mounted.
- Measure the adapter `RX` with an oscilloscope (0 to 3.3 V) before connecting `GPIO21`.
- Measure servo torque at 5.1 V with the steering assembled; if insufficient, use a 6 V branch.
- Confirm that the adapter's internal regulator starts at 5.1 V (`PWR` LED).
- Install the voltage divider on `GPIO20` to monitor the battery.
- Test the 24-pin FPC extension with the car assembled and set camera height and tilt.
- Confirm whether the S3 camera is OV3660 or OV2640, and run a continuous three-minute test.

8. Mechanics

> **Status as of November 12, 2026.** Mechanics is led by Sebastian Josue Diaz Encarnacion. He delivered the **v1** (October 16) and **v2** (November 12) parts manuals, plus a report with his answers and corrections. v1 was tested and failed due to rear slip; **v2 has not been tested yet**. Data is tagged as **[CAD]**, **[measured]**, **[author's data]** or **[pending]**.



8.1 Chassis evolution

Stage	Date	What it was
v1	Oct 16, 2026	Yellow dual-shaft DC motor, 8 parts, 300 × 38 × 150 mm chassis [CAD]
Intermediate	No date	JGB37-520 with a 1:1 two-gear transmission; still slipped
v2	Nov 12, 2026	260 × 120 × 45.65 mm chassis [CAD], differential and rear bearings

Four things changed from v1 to v2: the drivetrain gained 10 × 15 × 4 bearings and a differential; the motor became the 12 V, 1000 rpm JGB37-520 with encoder (the one mounted today is 600 rpm); steering was redesigned, with 70° programmed on v1 and 62° on v2; and the structure improved without numerical detail in the report. The 70° of v1 has no support in the delivered CAD: its assembly has steering suppressed and comes from another folder.

8.2 Measurements, materials and mass

Measurement	Value	Source
Overall length	260 mm	[measured] by the author
Overall width	130 mm	[measured] assembled; the CAD chassis is 120 mm
Overall height	~80 mm	[measured]
Wheelbase	**163 mm**	[measured]
Ground clearance	34 mm	[measured]
Chassis (CAD)	260 × 120 × 45.65 mm	[CAD]
Material and base	3D-printed PLA, 4 mm base	[author's data], [CAD]
Wheel	Nominal Ø65 × 25 mm; CAD envelope 69 × 30 mm	[CAD]; measure the real one
Total mass	**not weighed**	[pending]
Center of gravity	**not measured**	[pending]

The v1 CAD reports 377.7 g of parts and 705.6 g of assembly, but with no material assigned and **it is not a usable estimate**: the wheel comes out at 7.8 g/cm³ and the chassis at 1.0 g/cm³. v2 was not weighed either. The 1.5 kg limit and MIC/IT need a scale, not the CAD.

8.3 Steering and rear drivetrain

- Chain: **MG90S servo** → **servo arm** → **steering bar** → **tie rod** → **knuckles** [CAD, inferred]. The bar and the tie rod share a 50 mm hole spacing.
- The servo arm has **5 holes at 2.5 mm pitch**: the closest give less travel and more force; the farthest, the opposite. Programmed angle: **62° on v2**. **[pending]** real wheel angle, turning radius and toe.
- **Motor:** JGB37-520, 12 V, 1000 rpm in the v2 design, with encoder. Chain: motor → 6 mm shaft → `gear3` pinion (Ø26.8) → `gear_4` ring (Ø45.4) → differential (`Base_of_gear` , `gear1` / `gear2`) → `Axis_of_wheel` → wheels [CAD, inferred].
- **Ratio ~1.7:1** between `gear3` and `gear_4`, calculated from outer diameters. The files do not store tooth count, module or pressure angle: it must be confirmed.
- **Bearings:** 10 × 15 × 4 in the rear drivetrain; the `Axis_of_wheel` shaft is Ø9.8 and fits the Ø10 bearing. The differential is mechanical, not two motors.
- **Author's correction:** the v2 manual assumes the bearings are in the front knuckles; Sebastian Josue Diaz Encarnacion corrects that they go in the **rear drivetrain**. The CAD still shows front bearing seats: the discrepancy must be resolved before manufacturing.

In software this has a consequence: the firmware assumes a **solid rear axle** and a single encoder. With a differential, the two wheels can turn at different speeds; odometry and the IT loop must be validated on the real car.

8.4 Mounts

- **Servo:** two M2 screws in the ears; the arm is mounted on the spline and secured with the center screw.
- **Motor:** bolted to a flat base; the chassis has a Ø37 mm cylindrical cradle with a 3 mm wall.
- **LiDAR and IMU:** at **60 mm** height, mounted flat (0°).
- **Camera:** at the front, separated from the board by the 24-pin FPC extension. **[pending]** height and tilt.
- **Battery:** the CAD models it as 103 × 34 × 24 mm; electrical dimensions are not in the CAD.
- **Computer:** the file is named `Esp32p4`, but the model inside is a **Raspberry Pi 4B** (85 × 56 mm). It must be renamed or the model replaced.

8.5 Testing and v1 vs v2 comparison

Aspect	v1	v2
Motor	Yellow DC, no encoder	JGB37-520, 12 V, 1000 rpm, with encoder
Drivetrain	No differential; slipped	Differential and rear bearings
Servo angle	70° programmed	62° programmed
Chassis (CAD)	300 × 38 × 150 mm	260 × 120 × 45.65 mm
Mass	Not weighed	Not weighed
Tested?	Yes; rear slip failure	**No**

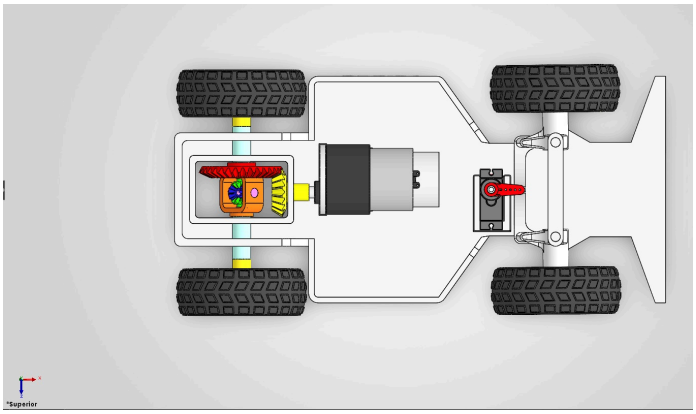
There are no measurements of weight, stiffness, torsion, slip or wheel angles. Therefore **there is no real numerical comparison** between v1 and v2: the improvement is in design, not in measured results. The differential is mechanical, and it is worth **confirming with the official text** that the rules allow it.

8.6 What is missing and what software needs

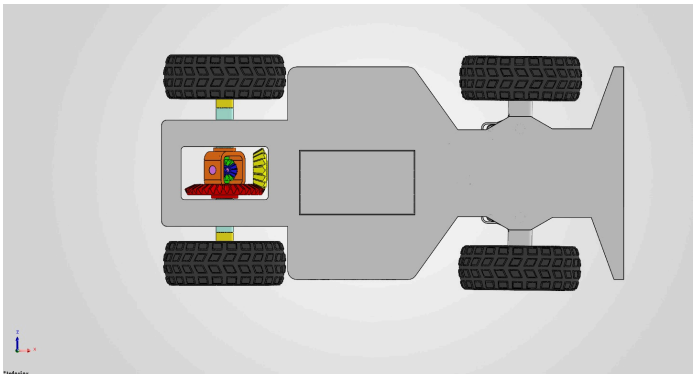
Item	Firmware today	Mechanical document	Action
Wheelbase	184.5 mm	**163 mm** [measured]	Fix `planner.wheelbase_cm`
Width	130 mm	130 mm assembled / 120 mm CAD	Confirm `chassis_width_cm`
Wheel	66 mm	65 nominal / 69 CAD	Measure the real one; affects odometry
Steering angle	40°	62° programmed	Measure real angle and turning radius
Mass and CG	Target ≤ 1.2 kg / 40 mm assumed	Not weighed / not measured	Weigh and measure
Rear axle	Solid	Differential	Validate odometry and MIC
Servo	SCS0009	MG90S	Reconcile electronics/mechanics
LiDAR	D500/D6	LD19	Reconcile model and protocol
Camera	Assumed height	Front, no dimensions	Measure height and tilt

8.7 CAD v2 views

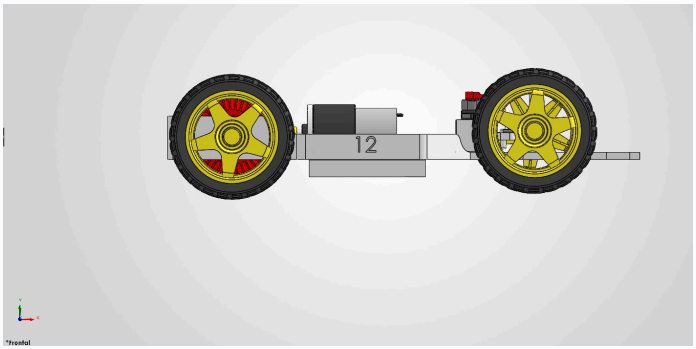
These are **SolidWorks renders of the v2 model**, not photographs of the assembled car. File names follow the SolidWorks views (`Frontal`, `Posterior`, `Derecha`, `Izquierda`), which do not match the car orientation: since the car is long on the X axis, `front` and `rear` are **side** views and `left` and `right` are **end** views.



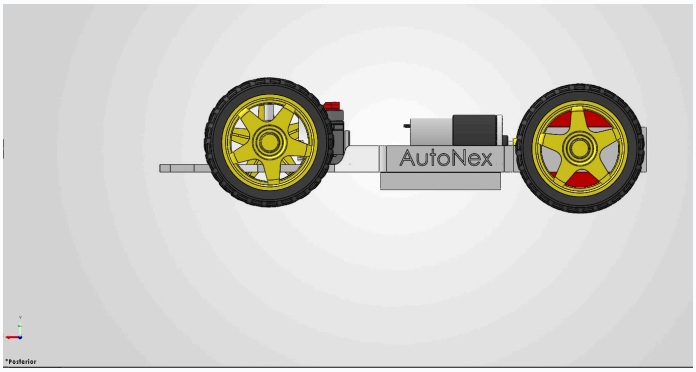
Top view: differential and ring gear at the rear, motor in the center, servo and steering bar at the front.



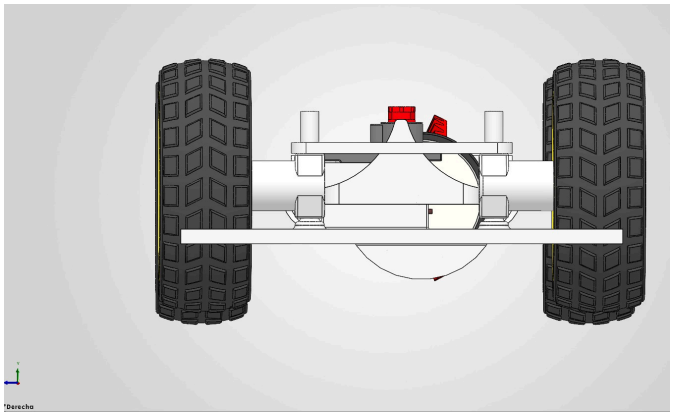
Bottom view: differential housing window and rectangular cutout in the base.



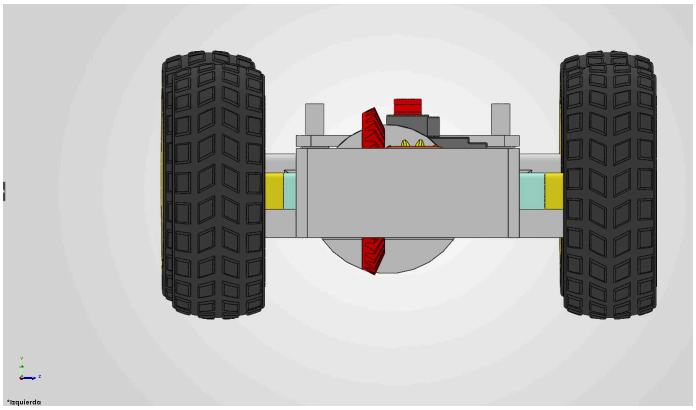
Side with the engraved number 12 (front.jpg, "Frontal" in SolidWorks).



Opposite side with "AutoNex" engraved (rear.jpg, "Posterior" in SolidWorks).



View from the nose (right.jpg, "Derecha" in SolidWorks), with the flat front plate.



View from the rear (left.jpg, "Izquierda" in SolidWorks), with the differential housing.

9. Tests and results

9.1 Methodology

- Every software change first goes through C tests and the simulator.
- Full missions are measured with `measure_mission.js` on **feasible** tracks, and new seeds are used at the end.
- A result is not fixed until it holds on two different benches. Seeds 1–8 were used for tuning and became overfitted; validation starts at 21.
- In electronics, each block is tested separately before connecting the next one.

9.2 Software results (simulation)

Test	Result
Full mission, feasible tracks, 3 signs, 24 rounds	**24/24 with 62 points and 0 contacts**, 118.9 s average
Parking entry, seeds 1–8	2–5 passes, ~4 average, 16–17 s, 0 failures
Parking entry, seeds 9–32	5 failures out of 56 (9 %, previously 17 %)
Lap 1 (mapping) at 34 cm/s	19.4 s vs. 29.6 s at the previous speed
IT on full mission	12/12 with 62 points (11/12 before the last fix)
IT + MIC	7 full rounds, 62 points, 0 contacts, 151 cm/s top speed
100-track bench (earlier stage)	99/100 in tuning and 95/100 in validation, 0 contacts
Localizer / `trace_map` accuracy	2.3 cm mean and 3.5 cm max / 0.78 cm RMS

9.3 Hardware results

The only part tested on real hardware is the S3 camera. Motors, encoders, LiDAR, IMU, servo and the flash map are integrated into the firmware but **not tested on the car**. Power electronics is documented and being assembled, with final measurements pending. The v1 chassis was tested and failed due to rear slip; v2 **is still untested**, without weighing or measuring CG or real angle.

9.4 Risks when moving to the real car

1. **Servo:** the simulator models an MG90S; the chosen servo is an SCS0009 and its real values must be measured.
2. **Grip:** μ, weight distribution and CG height are assumptions. If real grip is ≤ 0.40, IT fails.
3. **Single encoder:** the simulator averages two wheels; slip and effective diameter change odometry.
4. **LiDAR:** it does not emit `blind` or points closer than 5 cm; an obstacle against the nose does not appear.
5. **Actuators:** the car receives discrete 100 ms commands and the real servo has stops.
6. **Plan time and flash map:** not measured on the P4; the plan is sliced into 10 ms chunks.

10. Problems and lessons

Problem	Diagnosis	Change	Result
120 s stall at the lap-3 corner	93° heading treated as "behind"	Between 90° and 135°, request 85° to the short side	24/24 without stalls
Contact with the stop while parking	Arc check was stricter than the plan	Margin from 0.5 to 0.75 cm	0 contacts on seeds 21–32
"1 in 3 ends in `wrong-side`"	The tool drew infeasible tracks	`measure_mission.js` uses `feasible` by default	22/24 with 62 points
Parking stayed `stalled`	The pose overlapped the bay without clearance	Nearest free pose (2 cm, 6°)	Unblocked
IT lost distance at 150 cm/s	`odometry.c` discarded samples > 150 cm/s	Limit raised to 300 cm/s	Localizer stops lagging
10 cm lateral error at 150 cm/s	The follower ran at 10 Hz	Fast loop at 50 Hz	4.7 cm lateral error
At 35 cm/s the team's track failed	`plano_corregido.json` ends in `wrong-side`	Mapping speed set to 34 cm/s	Clean at 31, 33 and 34
The judge waited 15 s after parking	Inherited fixed delay	3 s and stop when the car halts	Less dead time

What we take away: a result counts when it repeats on new seeds and two benches; measurement tools also fail; every figure needs its configuration next to it; in electronics, testing by blocks and disconnecting the last addition isolates the error; and discarded ideas are documented too.

11. Current status and roadmap

Area	Status as of Nov 12, 2026	Next step	Owner
Software in simulation	✅ Full mission 24/24, 62 points, 0 contacts	Maintain and expand tests	Eddy
IT on the real car	☐ Needs calibration	Measure servo, grip, motor and tracking error	Eddy + Sebastian
Parking, MIC, encoders and flash	🔄 Integrated, not tested on hardware	Test by blocks on the P4	Eddy
Power electronics	🔄 Documented and being assembled	Measure current, servo at 1 Mbit/s, `RX` and `PWR`	Yeremy
Chassis and steering	🔄 v2 designed, **untested**	Test v2, weigh and measure CG, radius and angle	Sebastian
Documentation	🔄 README and role documents	Complete mechanics and measurements	Whole team

The order we will follow:

1. Close mechanics: weights, real wheelbase, maximum angle and sensor mounting.
2. Measure the real servo (speed, delay, torque) and motor (speed vs. PWM, current).
3. Bench-test LiDAR, IMU, encoder, P4–S3 link, camera extension and flash map.
4. Calibrate odometry and localizer on the real car.
5. Enable MIC and then IT, only once the previous measurements are loaded.
6. Run a continuous three-minute test and final timing adjustment.

12. Rule compliance

Requirement	How it is met	Evidence
Ackermann steering	Servo and knuckles; no speed-difference turning	`ACKERMANN.md`, firmware; mechanical differential pending confirmation
Dimensions ≤ 300 × 200 × 300 mm	260 × 130 mm chassis, ~80 mm high	v2 CAD and Sebastian's report
Mass ≤ 1.5 kg	Target ≤ 1.2 kg	Pending measurement
Non-omnidirectional wheels	Conventional wheels	Mechanical design
Red → right, green → left	Rule in the planner and reactive controller	`README.md`, tests
Three laps and parking	Phase machine	`wro_controller.c`
No radio during the round	Wi-Fi and Bluetooth off; physical start	`hw_start.c`, `docs/ELECTRONICS.md`
Original work and repository	37 commits, original code, documents and CAD	GitHub

13. Project management and documentation

- **Repository:** GitHub `Eddym06/AutoNex`, `main` branch. 37 commits between November 6 and 13: 28 software, 3 electronics, 4 photographs and 2 mechanics (v2 document and CAD).
- **Language and flow:** repository and role documents in English; internal documentation and this logbook in Spanish. Small, descriptive commits with the test result in the message; important decisions are written in `ESTADO.md`, `ARQUITECTURA.md` or `PENDIENTES_PARQUEO.md`.
- **Reference documents:** `README.md`, `README.es.md`, `ARQUITECTURA.md`, `ESTADO.md`, `ACKERMANN.md`, `PENDIENTES_PARQUEO.md`, `docs/DESIGN.md`, `docs/ELECTRONICS.md`, `simulator/INTELLIGENT_TRAJECTORY.md`, `simulator/TRACE_MAP.md`, `Motor Control/MIC/README.md` and `Motor Control/MIC/CALIBRACION.md`.

- **Images:** eight files in `photos/`: seven v2 CAD renders and the team photo. **There are still no photographs of the assembled physical car.**

14. Conclusions

Software is closed in simulation, and that is the most mature part of the project: the full mission, parking and motor control have repeatable results and their own test bench. Electronics is documented and assembly advances block by block, with final measurements still pending. Mechanics is the least validated part: v2 is designed and modeled, but it has not driven or been weighed.

The next milestone is not adding features, but measuring. Weigh the car, confirm the wheelbase and real wheel angle, characterize the servo and motor, and bench-test the LiDAR, IMU, encoder and camera extension. With those figures we calibrate odometry, then MIC and finally IT. Until then, any track number would be an assumption.

What is clear is the method: simulate before soldering, test by blocks, require every result to hold on new seeds, and write down what was discarded. That way of working is what made it possible to reach 24 out of 24 without having driven the final version yet.

Last updated: November 13, 2026. The logbook is part of the role documentation in the repository.